

BUSINESS REQUIREMENTS DOCUMENT

AI Software Delivery Assistant

AI-Native, Multi-Agent Project Delivery Intelligence Platform

| Document Attribute | Details                                                               |
|--------------------|-----------------------------------------------------------------------|
| Document Type      | Business Requirements Document (BRD)                                  |
| Version            | 1.0                                                                   |
| Status             | Draft for Build                                                       |
| Prepared For       | Product Owner, AI Engineering Team, Solution Architect, QA and DevOps |
| Primary Audience   | AI Developer / Full-Stack Developer / Technical Lead                  |
| Date               | 19 July 2026                                                          |
| Confidentiality    | Internal / Client Demonstration Use                                   |

**Purpose:** Provide step-by-step, build-ready requirements for an enterprise AI assistant that monitors software delivery, identifies risks, and prepares actionable project intelligence.

## 1. Document Control

| Version | Date        | Author/Owner                        | Change Summary                           | Approval |
|---------|-------------|-------------------------------------|------------------------------------------|----------|
| 0.1     | 19-Jul-2026 | Product Owner                       | Initial structure and concept definition | Pending  |
| 1.0     | 19-Jul-2026 | Product Owner / Delivery Leadership | Build-ready BRD                          | Pending  |

## 2. Executive Summary

The AI Software Delivery Assistant is an AI-native enterprise application designed to reduce the manual effort required to monitor software projects, prepare status reports, detect delivery risks, and coordinate follow-up actions. It will connect to delivery systems such as Azure DevOps or Jira, source-code repositories, test-management tools, knowledge repositories, and communication platforms.

The solution will use an orchestrator agent and specialized delivery agents for business analysis, development, quality assurance, DevOps, and project management. It will retrieve project evidence, reason over current and historical data, produce traceable recommendations, and require human approval before performing write actions.

| Business Outcome                                     | Target                                                                                        |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Reduction in daily project-status preparation effort | From 3-4 hours to 15-30 minutes per project manager                                           |
| Earlier risk visibility                              | Identify probable schedule, quality, dependency, or deployment risks before formal escalation |
| Consistent reporting                                 | Generate evidence-backed daily, weekly, sprint, and executive summaries                       |
| Improved accountability                              | Extract owners, actions, due dates, blockers, and pending approvals                           |
| Controlled automation                                | Use role-based access, audit logs, citations, approval gates, and safe tool usage             |

## 3. Business Problem

Project delivery information is fragmented across work-management tools, source repositories, testing systems, deployment pipelines, documents, emails, and chats.

Project managers and delivery leaders spend significant time collecting information, validating progress, identifying blockers, preparing reports, and following up with teams.

Existing dashboards show metrics but often do not explain why a project is at risk, what evidence supports the conclusion, or what action should be taken next.

## 4. Product Vision

Provide every delivery manager with a secure AI co-pilot that understands the project context, continuously assembles evidence, explains project health, and recommends practical next actions without replacing human accountability.

## 5. Business Objectives and Success Metrics

| ID    | Objective                           | Measure                           | Initial Target         |
|-------|-------------------------------------|-----------------------------------|------------------------|
| BO-01 | Automate project-health preparation | Time spent preparing daily status | At least 70% reduction |

| ID    | Objective                | Measure                                              | Initial Target                 |
|-------|--------------------------|------------------------------------------------------|--------------------------------|
| BO-02 | Improve risk detection   | Risks detected before missed milestone or escalation | At least 60% of material risks |
| BO-03 | Improve report quality   | Reports with traceable source evidence               | 100% of material assertions    |
| BO-04 | Improve action tracking  | Actions with owner and due date                      | At least 95%                   |
| BO-05 | Maintain safe operations | Unauthorized or unapproved write actions             | Zero                           |
| BO-06 | Control AI quality       | Critical hallucinations in accepted reports          | Less than 1%                   |

## 6. Scope

### 6.1 In Scope

- User authentication through Microsoft Entra ID or equivalent enterprise identity provider.
- Project selection and project-specific access control.
- Read integration with Azure DevOps or Jira, GitHub or Azure Repos, test-management data, CI/CD status, SharePoint or Confluence, Teams or email summaries.
- AI-generated project-health summary, sprint summary, risk analysis, action list, stakeholder update, and evidence links.
- Multi-agent orchestration with PM, BA, Development, QA, and DevOps analysis.
- Human approval before creating or updating tickets, sending messages, or changing delivery systems.
- Audit trail covering prompts, tool calls, retrieved evidence, model outputs, approvals, and actions.
- Evaluation framework for groundedness, completeness, risk precision, and report usefulness.

### 6.2 Out of Scope for MVP

- Autonomous code merge or production deployment.
- Automatic employee performance scoring.
- Financial invoicing, payroll, or contract decisions.
- Unrestricted reading of all organizational email or chat content.
- Replacing formal project governance or customer approvals.
- Training a foundation model from scratch.

## 7. Users and Personas

| Persona                            | Primary Need                                                                      | Allowed Actions                                                          |
|------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Delivery Manager / Project Manager | Understand project health, risks, actions, milestones, and reporting              | Read all authorized project evidence; approve or reject proposed actions |
| Technical Lead                     | Review development progress, code risks, dependencies, and technical debt         | Read engineering data; comment on recommendations                        |
| QA Lead                            | Review coverage, execution, defects, leakage risk, and release readiness          | Read QA evidence; validate quality findings                              |
| DevOps Lead                        | Review builds, deployments, environments, incidents, and rollback readiness       | Read pipeline and environment data                                       |
| Executive / Account Leader         | Receive concise portfolio or project-level business summary                       | Read approved summaries only                                             |
| Administrator                      | Configure connectors, permissions, policies, evaluation thresholds, and retention | Administrative functions with audit trail                                |

## 8. Primary Use Case: Daily Project Health

Trigger: The project manager selects a project and asks, “Give me today’s project health,” or a scheduled job prepares a draft report for review.

1. Authenticate the user and verify project access.
2. Load project configuration, reporting period, sprint dates, and source connectors.
3. Orchestrator decomposes the request into BA, Development, QA, DevOps, and PM analysis tasks.
4. Each agent retrieves only the minimum evidence required from authorized tools.
5. Agents return structured findings with evidence references, timestamps, confidence, and missing-data warnings.
6. Orchestrator reconciles contradictions, removes duplicates, and calculates overall health.
7. Evaluation layer checks grounding, data freshness, completeness, policy compliance, and unsupported claims.
8. Application displays the draft report with health, progress, risks, blockers, actions, decisions required, and evidence.
9. User edits or approves the report.
10. Any outbound action, such as updating a ticket or sending a stakeholder note, requires explicit user approval and is logged.

## 9. Functional Requirements

| ID     | Capability             | Requirement                                                                                                                                        | Priority |
|--------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-001 | Authentication         | The system shall authenticate users using enterprise SSO.                                                                                          | Must     |
| FR-002 | Authorization          | The system shall enforce project-level and connector-level role-based access.                                                                      | Must     |
| FR-003 | Project Configuration  | An administrator shall configure project name, tools, sprint calendar, repositories, environments, stakeholders, and reporting templates.          | Must     |
| FR-004 | Natural-Language Query | A user shall ask questions about authorized project delivery data.                                                                                 | Must     |
| FR-005 | Orchestration          | The orchestrator shall create a task plan and delegate work to specialist agents.                                                                  | Must     |
| FR-006 | BA Analysis            | The BA agent shall analyze requirements status, acceptance criteria, pending clarifications, scope changes, and dependencies.                      | Should   |
| FR-007 | Development Analysis   | The development agent shall analyze work-item progress, pull requests, commit activity, code review status, build quality, and technical blockers. | Must     |
| FR-008 | QA Analysis            | The QA agent shall analyze test coverage, execution, defect trends, severity, reopen rate, blocked testing, and release readiness.                 | Must     |
| FR-009 | DevOps Analysis        | The DevOps agent shall analyze build, deployment, environment, incident, and rollback evidence.                                                    | Must     |
| FR-010 | PM Analysis            | The PM agent shall analyze milestone status, capacity, dependencies, risks, decisions, and stakeholder commitments.                                | Must     |
| FR-011 | Evidence Citation      | Every material factual statement shall include a source reference and retrieval timestamp.                                                         | Must     |
| FR-012 | Freshness Warning      | The system shall flag stale, incomplete, inaccessible, or contradictory data.                                                                      | Must     |
| FR-013 | Health Scoring         | The system shall calculate configurable scope, schedule, quality, delivery, and dependency health.                                                 | Must     |
| FR-014 | Risk Detection         | The system shall identify risks, probability, impact, evidence, affected milestone, recommended mitigation, and owner.                             | Must     |
| FR-015 | Action Extraction      | The system shall produce actions with owner, due date, priority, status, and source.                                                               | Must     |
| FR-016 | Report Generation      | The system shall generate daily, weekly, sprint, release-readiness, and executive reports.                                                         | Must     |
| FR-017 | Human Approval         | The system shall require explicit approval before all write operations.                                                                            | Must     |

| ID     | Capability          | Requirement                                                                                                                       | Priority |
|--------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-018 | Ticket Update       | After approval, the system may create or update authorized work items.                                                            | Could    |
| FR-019 | Message Drafting    | The system shall draft, but not automatically send, stakeholder communications in the MVP.                                        | Should   |
| FR-020 | Feedback            | Users shall rate findings and mark them correct, incorrect, irrelevant, or missing context.                                       | Must     |
| FR-021 | Audit Trail         | The system shall log user request, retrieved sources, tool calls, model version, output, evaluations, approval, and write action. | Must     |
| FR-022 | Conversation Memory | The system may retain approved project preferences and prior summaries subject to retention rules.                                | Should   |
| FR-023 | Export              | Users shall export approved reports to PDF, Word, email draft, or collaboration platform.                                         | Should   |
| FR-024 | Administration      | Administrators shall manage prompts, model routing, connector secrets, policies, thresholds, templates, and retention.            | Must     |

## 10. Agent Responsibilities and Output Contracts

| Agent             | Inputs                                                                         | Required Analysis                                                               | Structured Output                                                           |
|-------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Orchestrator      | User intent, project config, time range                                        | Plan tasks, select tools, sequence agents, resolve conflicts, assemble response | Task plan, delegated tasks, consolidated report, unresolved gaps            |
| BA Agent          | Backlog, requirements, acceptance criteria, change requests, meeting decisions | Readiness, ambiguity, dependency, scope change, approval status                 | Requirement findings, missing acceptance criteria, pending decisions, risks |
| Development Agent | Stories, tasks, repositories, PRs, builds, code quality                        | Progress, blocked work, review delay, build health, technical debt              | Development health, evidence, risks, actions                                |
| QA Agent          | Test cases, runs, defects, coverage, environments                              | Coverage, pass rate, blocked tests, severity trends, leakage risk               | QA health, release concerns, defects requiring action                       |
| DevOps Agent      | Pipelines, deployments, incidents, environments                                | Build/deploy reliability, environment readiness, rollback status                | Operational health, failed stages, release blockers                         |
| PM Agent          | Milestones, sprint, capacity, dependencies, actions, decisions                 | Schedule variance, forecast, dependency risk, stakeholder commitments           | Overall project interpretation, priorities, proposed communication          |

### 10.1 Mandatory Agent Response Schema

Each specialist agent must return machine-readable JSON conforming to the following logical structure:

```
{
  "agent": "qa_agent",
  "status": "complete | partial | failed",
  "summary": "Concise evidence-based finding",
  "health": "green | amber | red | unknown",
  "confidence": 0.0,
  "findings": [{
    "title": "Blocked regression execution",
    "severity": "high",
    "evidence": [{ "source": "Test Management", "record_id": "TR-123", "retrieved_at": "ISO-8601" }],
    "impact": "Release readiness",
    "recommendation": "Restore environment and rerun critical suite",
    "owner": "QA Lead",
    "due_date": "YYYY-MM-DD"
  }],
  "data_gaps": [],
  "policy_flags": []
}
```

## 11. Business Rules

| Rule ID | Rule                                                                                                                                                               |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BR-01   | No claim shall be presented as fact without retrieved evidence or an explicit “inference” label.                                                                   |
| BR-02   | No specialist agent may access a source unless the user and project configuration authorize it.                                                                    |
| BR-03   | No write action shall be executed without an approval record tied to the exact proposed payload.                                                                   |
| BR-04   | Health shall be “unknown” when minimum required data is unavailable; the system shall not guess.                                                                   |
| BR-05   | The latest source event timestamp shall be shown for all time-sensitive metrics.                                                                                   |
| BR-06   | Conflicting evidence shall be displayed with an explanation; the system shall not silently choose one.                                                             |
| BR-07   | Personal performance conclusions shall be excluded from MVP reports.                                                                                               |
| BR-08   | The system shall minimize data retrieval and avoid sending secrets, source code, or sensitive data to models unless explicitly approved and technically protected. |
| BR-09   | Model output shall be treated as a draft until evaluation checks pass.                                                                                             |
| BR-10   | A user correction shall not overwrite source-system evidence; it shall be stored as feedback or approved context.                                                  |

## 12. Project Health Model

For the MVP, use configurable deterministic scoring supported by AI explanation. Do not allow the LLM alone to assign the final score.

| Dimension            | Example Inputs                                           | Default Weight | Red Trigger Examples                                                 |
|----------------------|----------------------------------------------------------|----------------|----------------------------------------------------------------------|
| Scope                | Story readiness, scope changes, pending decisions        | 20%            | Critical requirement unresolved; unapproved scope expansion          |
| Schedule             | Planned vs completed work, milestone trend, blockers     | 25%            | Critical milestone forecast missed by more than configured tolerance |
| Quality              | Test pass rate, critical defects, coverage, reopen trend | 25%            | Open release-blocking defect; critical suite not executed            |
| Engineering          | PR age, build status, code quality, dependencies         | 15%            | Main build failed; critical dependency unresolved                    |
| Release / Operations | Deployment, environment, incident, rollback readiness    | 15%            | Production-like environment unavailable; rollback not validated      |

## 13. Non-Functional Requirements

| ID     | Area         | Requirement                                                                                       |
|--------|--------------|---------------------------------------------------------------------------------------------------|
| NFR-01 | Performance  | Standard daily-health response should complete within 60 seconds for configured MVP data volumes. |
| NFR-02 | Availability | Target 99.5% monthly availability excluding planned maintenance.                                  |
| NFR-03 | Security     | Use enterprise SSO, RBAC, managed identities, secret vault, encryption in transit and at rest.    |
| NFR-04 | Privacy      | Support configurable retention, data minimization, masking, and deletion.                         |
| NFR-05 | Auditability | All model and tool interactions shall be traceable by correlation ID.                             |
| NFR-06 | Scalability  | Support horizontal scaling of API, workers, and agent execution.                                  |

| ID     | Area            | Requirement                                                                                         |
|--------|-----------------|-----------------------------------------------------------------------------------------------------|
| NFR-07 | Reliability     | Use retry, timeout, circuit breaker, idempotency, and partial-result handling.                      |
| NFR-08 | Explainability  | Show evidence, assumptions, confidence, and calculation details.                                    |
| NFR-09 | Accessibility   | Target WCAG 2.1 AA for the web application.                                                         |
| NFR-10 | Maintainability | Prompts, policies, templates, thresholds, and tool mappings shall be configurable.                  |
| NFR-11 | Observability   | Capture latency, token use, tool failures, retrieval quality, evaluator results, and user feedback. |
| NFR-12 | Portability     | Use provider abstractions so models and connectors can be replaced with limited code change.        |

## 14. Target Solution Architecture

Recommended logical components:

- Web UI: project selector, chat, health dashboard, evidence drawer, report editor, approval panel, feedback.
- Application API: authentication, authorization, project configuration, report management, approval workflow.
- Agent Orchestrator: task planning, agent invocation, state management, conflict resolution, and response assembly.
- Specialist Agents: BA, Development, QA, DevOps, PM.
- Tool Gateway / MCP Layer: standardized, permission-aware connectors to enterprise systems.
- Retrieval Layer: document ingestion, chunking, metadata filtering, hybrid search, and citations.
- Rules and Scoring Engine: deterministic metrics, thresholds, health calculation, and policy checks.
- Evaluation Layer: groundedness, completeness, contradiction, safety, freshness, and format validation.
- Data Stores: application database, vector index, short-term agent state, approved memory, audit log.
- Observability: logs, traces, metrics, token and cost tracking, model and tool performance.

### 14.1 Suggested Technology Stack

| Layer            | Suggested Option                                    | Implementation Note                                                                    |
|------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------|
| Frontend         | React / Next.js                                     | Use enterprise design system; stream agent progress without exposing hidden reasoning. |
| Backend          | Python FastAPI or .NET                              | Keep orchestration and domain services modular.                                        |
| AI Platform      | Azure AI Foundry or equivalent                      | Use approved enterprise model endpoints and content controls.                          |
| Orchestration    | Semantic Kernel, LangGraph, or custom state machine | Prefer explicit graph/state transitions over unconstrained agent loops.                |
| Tool Integration | MCP-compatible tool gateway plus direct APIs        | Wrap each tool with authentication, schema validation, timeout, and audit.             |
| Search / RAG     | Azure AI Search or equivalent hybrid index          | Filter by tenant, project, document type, and access rights.                           |
| Database         | PostgreSQL / Azure SQL                              | Store project config, reports, approvals, feedback, and audit metadata.                |
| Cache / State    | Redis                                               | Use for short-lived execution state and distributed locks.                             |
| Secrets          | Azure Key Vault                                     | Never store connector secrets in prompts or source control.                            |
| Observability    | Application Insights / OpenTelemetry                | Trace end-to-end request, agent, model, retrieval, and tool spans.                     |

## 15. Minimum Data Model

| Entity        | Key Fields                                                                                       |
|---------------|--------------------------------------------------------------------------------------------------|
| User          | user_id, identity_provider_id, role, status                                                      |
| Project       | project_id, name, owner, status, sprint_calendar, health_config                                  |
| ProjectAccess | user_id, project_id, role, permissions                                                           |
| Connector     | connector_id, project_id, type, endpoint, credential_reference, scopes, status                   |
| AgentRun      | run_id, user_id, project_id, request, plan, status, timestamps, model_version                    |
| ToolCall      | tool_call_id, run_id, connector_id, operation, request_hash, response_reference, status, latency |
| Evidence      | evidence_id, run_id, source_type, source_record_id, title, timestamp, access_uri, content_hash   |
| Finding       | finding_id, run_id, agent, severity, health, summary, confidence                                 |
| Risk          | risk_id, finding_id, probability, impact, owner, mitigation, due_date, status                    |
| Action        | action_id, source_finding_id, owner, due_date, priority, proposed_write_payload, approval_status |
| Report        | report_id, project_id, period, report_type, content, score, status, approved_by                  |
| Evaluation    | evaluation_id, run_id, evaluator_type, score, result, details                                    |
| Feedback      | feedback_id, finding_id, user_id, rating, category, comments                                     |
| AuditEvent    | event_id, actor, action, resource, correlation_id, timestamp, metadata                           |

## 16. Security, Governance, and Responsible AI

- Use least-privilege permissions for every connector.
- Separate read tools from write tools; disable write tools by default.
- Validate all tool arguments against strict schemas and approved resource scopes.
- Defend against prompt injection in retrieved documents by treating retrieved content as untrusted data, not instructions.
- Mask secrets, credentials, personal data, and restricted source code before model submission where required.
- Maintain immutable audit events for sensitive actions.
- Apply content filtering and organization-specific policies.
- Provide a kill switch per connector, project, model, or agent.
- Define data retention for prompts, retrieved text, outputs, embeddings, and logs.
- Conduct threat modeling before production rollout.

## 17. User Experience Requirements

| Screen                  | Minimum Components                                                                   |
|-------------------------|--------------------------------------------------------------------------------------|
| Home / Project Selector | Authorized projects, current health, latest report, outstanding approvals            |
| Assistant Workspace     | Question input, time period, run progress, cancel control, structured answer         |
| Health Dashboard        | Overall RAG status, dimension scores, trend, stale-data warning                      |
| Findings and Evidence   | Finding, severity, confidence, evidence links, timestamps, source preview            |
| Risks and Actions       | Risk, impact, probability, mitigation, owner, due date, approval state               |
| Report Editor           | Editable generated report, version history, approval, export                         |
| Approval Center         | Exact proposed action, target system, before/after payload, approve/reject           |
| Administration          | Projects, connectors, permissions, prompts, policies, thresholds, evaluation results |



## 18. Step-by-Step Build Instructions

### Phase 0 - Foundation and Decisions (1 week)

11. Confirm the MVP tool set: select one work-management tool, one source repository, one test source, and one document source.
12. Define one pilot project and obtain read-only service credentials.
13. Agree health dimensions, thresholds, report template, and user roles.
14. Create threat model, data classification, and approval policy.
15. Create repositories, coding standards, environments, CI pipeline, and issue backlog.

### Phase 1 - Application Skeleton (1-2 weeks)

16. Implement SSO and project-level RBAC.
17. Create project, connector, user-access, report, and audit database tables.
18. Build web shell with project selector and assistant workspace.
19. Implement correlation IDs, structured logging, error handling, and feature flags.
20. Add mock connectors and sample project data to unblock front-end and agent development.

### Phase 2 - Read Connectors and Evidence (2 weeks)

21. Implement read-only connectors for backlog/work items, repository/PR/build data, test results/defects, and project documents.
22. Normalize records into internal evidence schemas.
23. Add timestamp, source ID, deep link, project ID, sensitivity, and access metadata.
24. Implement timeouts, pagination, retries, rate-limit handling, and connector health checks.
25. Create contract tests with recorded non-sensitive fixtures.

### Phase 3 - Deterministic Metrics and Health (1-2 weeks)

26. Implement sprint progress, overdue items, blocked work, PR age, build status, test pass rate, critical defects, and deployment status.
27. Implement configurable thresholds and weighted health scoring.
28. Return unknown rather than green when required data is missing.
29. Write unit tests for every formula and threshold boundary.

### Phase 4 - Single-Agent MVP (2 weeks)

30. Build one Delivery Analyst agent before implementing multiple agents.
31. Provide the agent only approved read tools and a strict output schema.
32. Require evidence IDs for every finding.
33. Add prompt-injection defenses, maximum tool-call limits, and execution timeout.
34. Generate the daily-health report and show citations in the UI.

### Phase 5 - Multi-Agent Specialization (2-3 weeks)

35. Split the Delivery Analyst into BA, Development, QA, DevOps, and PM agents.
36. Implement an explicit orchestration graph with maximum steps and deterministic routing rules.
37. Run independent agent tasks in parallel where safe.
38. Implement contradiction detection and duplicate finding consolidation.
39. Do not expose private chain-of-thought; expose concise rationale and evidence only.

## Phase 6 - Evaluation and Quality Gates (2 weeks)

40. Create a gold dataset of realistic project snapshots and expected findings.
41. Implement groundedness, citation validity, freshness, completeness, schema, and policy evaluators.
42. Block report publication when critical evaluation checks fail.
43. Capture user feedback and compare against expected results.
44. Track quality by prompt version, model version, connector, and project.

## Phase 7 - Human Approval and Write Actions (1-2 weeks)

45. Add message drafting and work-item update proposal generation.
46. Display the exact target, operation, and payload before approval.
47. Bind approval to a payload hash so changed payloads require reapproval.
48. Implement idempotency and rollback or compensation where the target system supports it.
49. Log the approving user, timestamp, result, and source evidence.

## Phase 8 - Pilot and Production Hardening (2 weeks)

50. Run the solution with one project in shadow mode alongside the current manual process.
51. Compare AI reports to PM reports and classify misses and false positives.
52. Tune thresholds, prompts, tools, and report formatting.
53. Complete load, security, privacy, disaster recovery, and operational readiness testing.
54. Release to a limited user group and monitor adoption, quality, latency, and cost.

## 19. MVP User Stories and Acceptance Criteria

| ID    | User Story                                                                                                           | Acceptance Criteria                                                                                                                                                                        |
|-------|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| US-01 | As a project manager, I want to view today's project health so that I can focus on risks and decisions.              | Given an authorized project, when the user requests daily health, then the application displays overall and dimension health, progress, risks, blockers, actions, data gaps, and evidence. |
| US-02 | As a project manager, I want every critical claim linked to evidence so that I can trust and verify the report.      | Every material finding includes source system, record identifier, retrieval timestamp, and accessible link; unsupported claims fail evaluation.                                            |
| US-03 | As a QA lead, I want the assistant to highlight release blockers so that the team can act before deployment.         | Open critical defects, blocked critical tests, unavailable environments, or missing mandatory coverage are shown as release concerns according to configured rules.                        |
| US-04 | As a delivery manager, I want a stakeholder-update draft so that reporting is faster and consistent.                 | The system generates an editable summary based only on approved findings and does not send it automatically.                                                                               |
| US-05 | As an administrator, I want to control project access and connectors so that information is protected.               | A user without project access cannot query, retrieve, infer, or view that project's data.                                                                                                  |
| US-06 | As an approver, I want to review proposed changes before execution so that the AI cannot act without accountability. | Write action remains pending until explicit approval; executed payload matches approved payload hash; all events are audited.                                                              |
| US-07 | As a product owner, I want to measure AI quality so that we can safely improve it.                                   | Dashboard shows evaluator scores, user feedback, false-positive and false-negative samples, model/prompt version, latency, and cost.                                                       |

## 20. Test Strategy

| Test Type          | Coverage                                                                                      |
|--------------------|-----------------------------------------------------------------------------------------------|
| Unit               | Scoring rules, threshold boundaries, schemas, authorization helpers, payload hashing, parsers |
| Connector Contract | Authentication, pagination, rate limits, timeouts, malformed responses, access denied         |

| Test Type       | Coverage                                                                                          |
|-----------------|---------------------------------------------------------------------------------------------------|
| Integration     | End-to-end evidence retrieval, agent invocation, evaluation, report storage                       |
| AI Evaluation   | Groundedness, citation correctness, completeness, contradiction, risk precision, prompt injection |
| Security        | RBAC, tenant/project isolation, secret handling, injection, SSRF, tool misuse, audit integrity    |
| Performance     | Concurrent projects, large backlogs, long documents, connector latency, model rate limits         |
| Resilience      | Partial connector failure, stale data, model timeout, repeated request, write-action failure      |
| User Acceptance | PM usefulness, clarity, edit effort, missed risks, false alarms, workflow fit                     |
| Regression      | Gold dataset executed for every prompt, model, scoring, or connector change                       |

## 21. MVP Definition of Done

- SSO and project RBAC are operational and security-tested.
- At least four read-only source categories are integrated for one pilot project.
- Daily project-health report is generated with health, risks, actions, data gaps, and evidence.
- Every material statement is traceable to evidence or explicitly marked as inference.
- Deterministic scoring is implemented and unit-tested.
- Evaluation gates are executed and critical failures prevent publication.
- User feedback and audit trail are available.
- No write action occurs without explicit human approval.
- Operational dashboards show latency, failures, token usage, cost, evaluation scores, and connector health.
- Pilot acceptance criteria and responsible-AI review are approved.

## 22. Risks and Mitigations

| Risk                                  | Impact                          | Mitigation                                                                           |
|---------------------------------------|---------------------------------|--------------------------------------------------------------------------------------|
| Incomplete or stale source data       | Incorrect health or missed risk | Freshness checks, unknown state, data-gap warnings, connector monitoring             |
| Hallucinated finding                  | Loss of trust or bad decision   | Mandatory citations, structured output, evaluators, deterministic scoring            |
| Prompt injection in project documents | Tool misuse or data leakage     | Treat content as untrusted, instruction hierarchy, tool allowlists, output filtering |
| Excessive agent autonomy              | Uncontrolled actions or cost    | Explicit state graph, tool limits, timeouts, approval gates, budgets                 |
| Cross-project data leakage            | Confidentiality breach          | Project-scoped tokens, metadata filters, access checks at retrieval and display      |
| High false-positive risk alerts       | Alert fatigue                   | Configurable thresholds, feedback loop, gold dataset, precision tracking             |
| Connector instability                 | Partial or delayed reports      | Retries, circuit breakers, caching, partial result and status indication             |
| Model or token cost growth            | Poor unit economics             | Model routing, summarization, caching, budgets, cost monitoring                      |
| Overreliance on AI                    | Weak human governance           | Draft status, source verification, explicit ownership, approval workflow             |

## 23. Developer Deliverables

- Solution architecture and threat model.
- Source code with modular services, tests, infrastructure scripts, and CI/CD pipeline.
- Connector specifications and permission matrix.
- Prompt and agent configuration with version control.
- Database schema and migrations.
- API specification and internal tool schemas.
- Evaluation dataset, evaluator code, and quality dashboard.
- Deployment guide, runbook, support guide, and rollback plan.
- Demo script using a representative project snapshot.
- Known limitations and production-readiness checklist.

## 24. Recommended First Sprint Backlog

| Order | Backlog Item                                                                      | Expected Output                            |
|-------|-----------------------------------------------------------------------------------|--------------------------------------------|
| 1     | Create project repository, environments, CI, linting, unit-test framework         | Deployable empty service and UI            |
| 2     | Implement SSO and ProjectAccess authorization middleware                          | Authorized project selection               |
| 3     | Create core database schema and audit-event service                               | Persisted projects, runs, and audit events |
| 4     | Build mock connector framework and normalized Evidence model                      | Repeatable sample evidence                 |
| 5     | Implement project-health rules for sprint progress, blocked work, build and tests | Deterministic health JSON                  |
| 6     | Build assistant endpoint with one Delivery Analyst agent                          | Structured findings with evidence IDs      |
| 7     | Build basic UI for health, findings, evidence, risks, and gaps                    | End-to-end demo                            |
| 8     | Create initial 20-case evaluation dataset and automated checks                    | Baseline groundedness and schema score     |
| 9     | Add observability and cost telemetry                                              | Traceable end-to-end runs                  |
| 10    | Conduct sprint demo and capture PM feedback                                       | Prioritized Sprint 2 backlog               |

## Appendix A - Daily Project Health Report Template

### Overall Health and Executive Summary

[Generated content with evidence references]

### Progress and Milestones

[Generated content with evidence references]

### Scope and Requirement Readiness

[Generated content with evidence references]

### Development Health

[Generated content with evidence references]

## Quality and Release Readiness

[Generated content with evidence references]

## Build, Deployment, and Environment Health

[Generated content with evidence references]

## Top Risks

[Generated content with evidence references]

## Blockers and Decisions Required

[Generated content with evidence references]

## Actions by Owner and Due Date

[Generated content with evidence references]

## Data Gaps and Stale Sources

[Generated content with evidence references]

## Evidence References

[Generated content with evidence references]

## Appendix B - Sample End-User Request

*“Review Project Alpha for the current sprint. Tell me whether we are on track for Friday’s release, identify the top five risks and required decisions, and draft a concise stakeholder update. Use only evidence refreshed within the last 24 hours and clearly flag missing data.”*

## Appendix C - Implementation Guardrails for the AI Developer

- Build the deterministic data and scoring foundation before adding agent autonomy.
- Prefer explicit workflows and typed schemas over open-ended agent loops.
- Never place raw credentials or secrets inside prompts.
- Do not store private chain-of-thought; store concise explanations, evidence, and decisions.
- Make every failure visible: partial source, stale data, timeout, denied access, evaluation failure.
- Version prompts, tools, models, thresholds, and evaluation datasets.
- Design every write action as a proposed, reviewable, idempotent transaction.
- Use representative but non-sensitive test data in development.
- Treat evaluation and observability as product features, not post-production tasks.